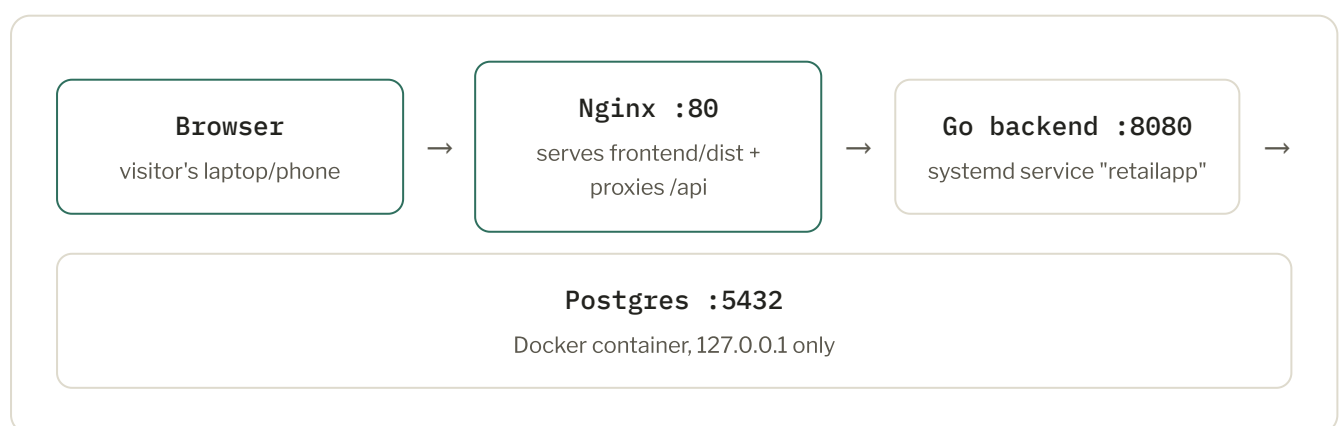


everything else listens on `localhost` only.



Only port 80 (and later 443) needs to be open in the EC2 security group. Ports 8080 and 5432 are never exposed to the internet — that's intentional.

1 Connect over SSH

From Windows PowerShell, in the folder holding the `.pem` key:

```
ssh -i "my-key.pem" ubuntu@<PUBLIC_IP>
```

Everything from here runs *inside* that SSH session, on the server — not on the local PC.

2 Install dependencies

```
sudo apt update
sudo apt install -y git nginx docker.io docker-compose-plugin
sudo systemctl enable --now docker
```

Go and Node, via the official installers (Ubuntu's own packages are too old):

```
# Go
curl -OL https://go.dev/dl/go1.23.4.linux-amd64.tar.gz
sudo rm -rf /usr/local/go
sudo tar -C /usr/local -xzf go1.23.4.linux-amd64.tar.gz
echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
source ~/.bashrc

# Node 20
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
```

3 Get the code

```
git clone <repo-url>
cd retailerapp-main
```

4 Start the database

```
echo "POSTGRES_PASSWORD=$(openssl rand -hex 16)" > .env
sudo docker compose --env-file .env up -d
sudo docker compose ps
```

The generated password lives in `.env` — it's referenced again in step 5 and step 8.

5 Build and run the backend

```
go build -o retailapp ./cmd/server
openssl rand -hex 32 # this is the session secret – copy it
```

Runs as a systemd service so it survives crashes and reboots:

```
sudo nano /etc/systemd/system/retailapp.service
```

```
[Unit]
Description=Retail App Backend
After=network.target docker.service

[Service]
Type=simple
User=root
WorkingDirectory=/root/retailerapp-main
ExecStart=/root/retailerapp-main/retailapp
Environment=DATABASE_URL=postgres://retailapp:<DB_PASSWORD>@localhost:5432/retai
Environment=SESSION_KEY=<SESSION_SECRET>
Restart=on-failure

[Install]
WantedBy=multi-user.target
```



```
sudo systemctl daemon-reload
sudo systemctl enable --now retailapp
sudo systemctl status retailapp
curl http://localhost:8080/health # should print: ok
```

Match User to the folder owner. If the `/root/...` , `User=` must be `root` — `ubuntu` c

6 Build the frontend

```
cd frontend
npm ci
npm run build
cd ..
```

Output lands in `frontend/dist` — plain static files, that's all Nginx needs to serve.

7 Configure Nginx

```
sudo nano /etc/nginx/sites-available/retailapp
```

```
server {
    listen 80;
    server_name _;

    root /root/retailerapp-main/frontend/dist;
    index index.html;

    location /api/ {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
    location /health {
        proxy_pass http://127.0.0.1:8080;
    }
    location / {
        try_files $uri /index.html;
    }
}
```

```
sudo rm -f /etc/nginx/sites-enabled/default
sudo ln -s /etc/nginx/sites-available/retailapp /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
chmod 755 /root # nginx runs as www-data — needs to traverse into /root
```

8 Load the schema, create the first login

A fresh database has no tables and no users yet — the login page loads, but nothing can sign in until both exist.

```
sudo docker exec -i $(sudo docker compose ps -q db) \
  psql -U retailapp -d retailapp < migrations/000001_init_schema.up.sql

go build -o bootstrap ./cmd/bootstrap
DATABASE_URL="postgres://retailapp:<DB_PASSWORD>@localhost:5432/retailapp?sslmod
```

Answers three prompts — name, username, password — then prints `Admin user created:`
... That username/password is what you log in with at `http://<PUBLIC_IP>` .

Gotchas actually hit this run

SYMPTOM	CAUSE	FIX
<code>status=203/EXEC</code>	systemd couldn't find/execute the binary — <code>WorkingDirectory</code> / <code>ExecStart</code> pointed at a path that didn't exist	Match the paths to <code>pwd</code> in the actual repo folder
<code>status=200/CHDIR</code>	<code>User=ubuntu</code> but the repo sits in <code>/root/...</code> , which is mode 700 — <code>ubuntu</code> has zero access	Set <code>User=root</code> when the repo lives under <code>/root</code>
Service starts but env var missing	<code>Environment=<value></code> written without the <code>NAME=</code> prefix — systemd has no idea what the value is for	Always <code>Environment=SESSION_KEY=<value></code> , never bare
Nginx 403 Forbidden	<code>www-data</code> can't traverse into <code>/root</code> to read <code>frontend/dist</code>	<code>chmod 755 /root</code>

Command cheatsheet

WANT TO...	RUN
Check backend is alive	<code>curl http://localhost:8080/health</code>
Restart backend	<code>sudo systemctl restart retailapp</code>
Tail backend logs	<code>sudo journalctl -u retailapp -f</code>
Backend status	<code>sudo systemctl status retailapp</code>
Database status	<code>sudo docker compose ps</code>
Reload Nginx after config edit	<code>sudo nginx -t && sudo systemctl reload nginx</code>
Open a psql shell	<code>sudo docker exec -it \$(sudo docker compose ps -q db) psql -U retailapp -d retailapp</code>

Redeploying after a code change

Pull the new code, then rebuild only what changed:

Before this is truly public